COMP1801 - Machine Learning Coursework Report

Dishan Bhandari – 001483674

Word Count: 3444

1. Executive Summary

This report was made to provide strong arguments and comparisons between two types of Machine Learning models: Regression and Classification, to address a business problem. The core task in hand was determining whether a metal part based on its lifespan could be considered safe for use or not. Explanatory Data Analysis on the dataset provided outline of the data with different visual plots to support findings. Based on these findings, two candidate models were considered for experimentation to ensure best selection. A well-defined pre-processing pipeline consisting proper encoding and scaling methods ensured that proper data of same split ratio was fed to train models ensuring parity and avoid any kind of bias. Evaluation was done by researching the performance metrics that best describe the model's output and interpretations. Metrics were different with respect to the tasks but same within, for fair model comparisons. Hyperparameters were selected using different tuning frameworks to provide optimal solutions. Finally, after analysing all relevant aspects, a Random Forest Classification model was recommended based on its capability, interpretability and scalability, maximizing business value and best addressing the use-case scenario of the company.

2. Data Exploration

The dataset was imported using the Pandas read_csv() method, the head() method and the shape attribute showed that the dataset has 1000 observations and 16 features. The info() method provides the datatype of each columns consisting both numerical and categorical columns. The describe() method generates statistical summaries like mean, standard deviation, and percentiles for the numeric columns.

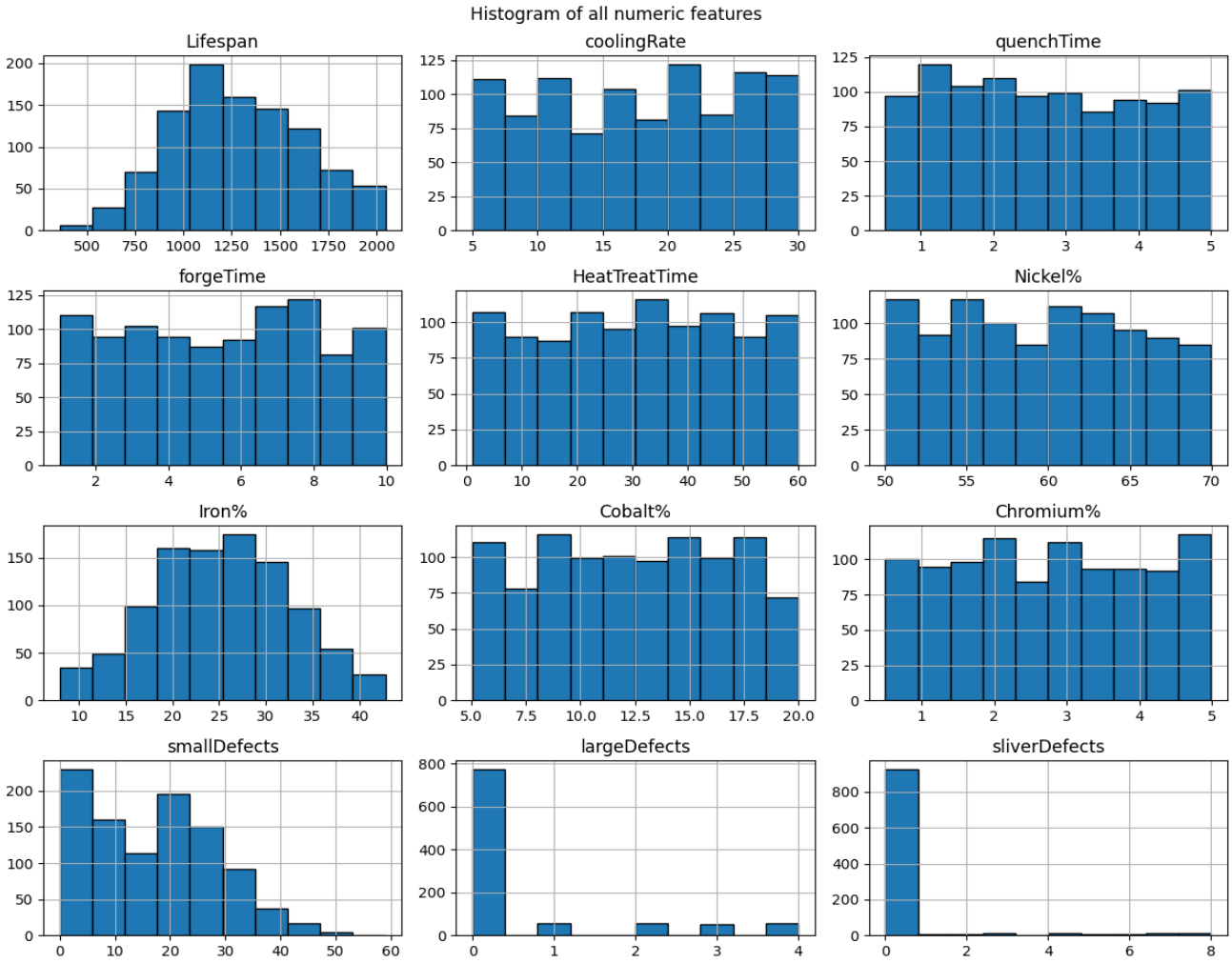

fig- 1: Histogram plots of all numeric columns

Histogram plots aid in Univariate analysis, from fig-1 plots of ‘smallDefects’, ‘largeDefects’ and ‘sliverDefects’ stand out. Since these columns have right-skewed distribution (Positive Skewness), and presence of zero values, different transformations were tested to make it symmetric as shown in fig-2. Log transformation was not possible as columns had zero values (Islam, 2023).

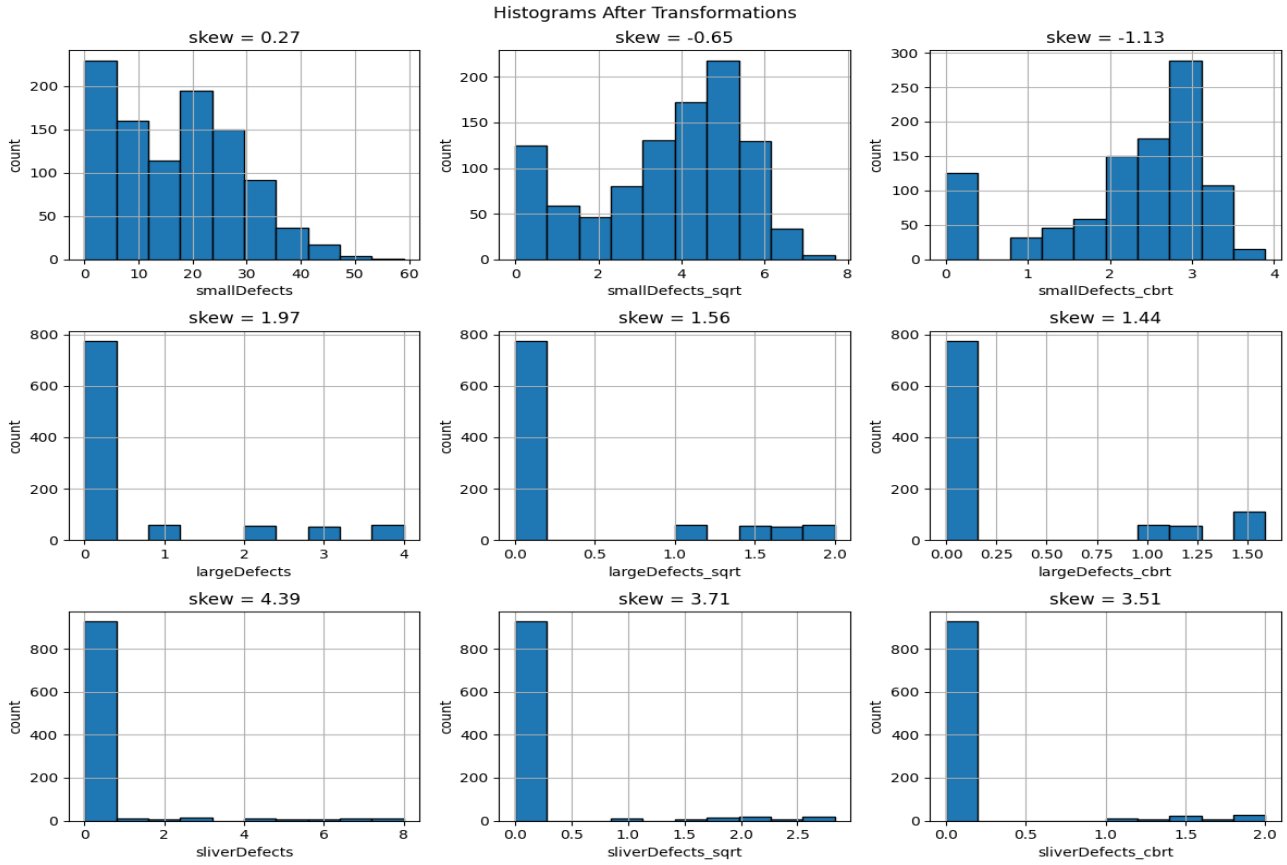

fig- 2: Histogram plots showing square root and cube root transformations

From fig-2, it's evident that 'smallDefects' column post-transformation gave worse results so, it was better to leave it untransformed. For other two columns, the skew was still high but cube-root transformation gave the best results.

Using value_counts(), 'sliverDefects' was found to contain 92.7% zero values, indicating extreme sparsity and potentially uninformative so, it was dropped. In contrast, 'largeDefects' had 77.5% zeros, still sparse but likely to contain valuable information. To retain this information, cube-root transformation was applied to create 'largeDefects_cbrt' column, and the original column was dropped.

For categorical columns, Count plots were made to check data balance and possible bias showing no clear dominant values, and boxplots for bivariate analysis with the target shown by fig-3.

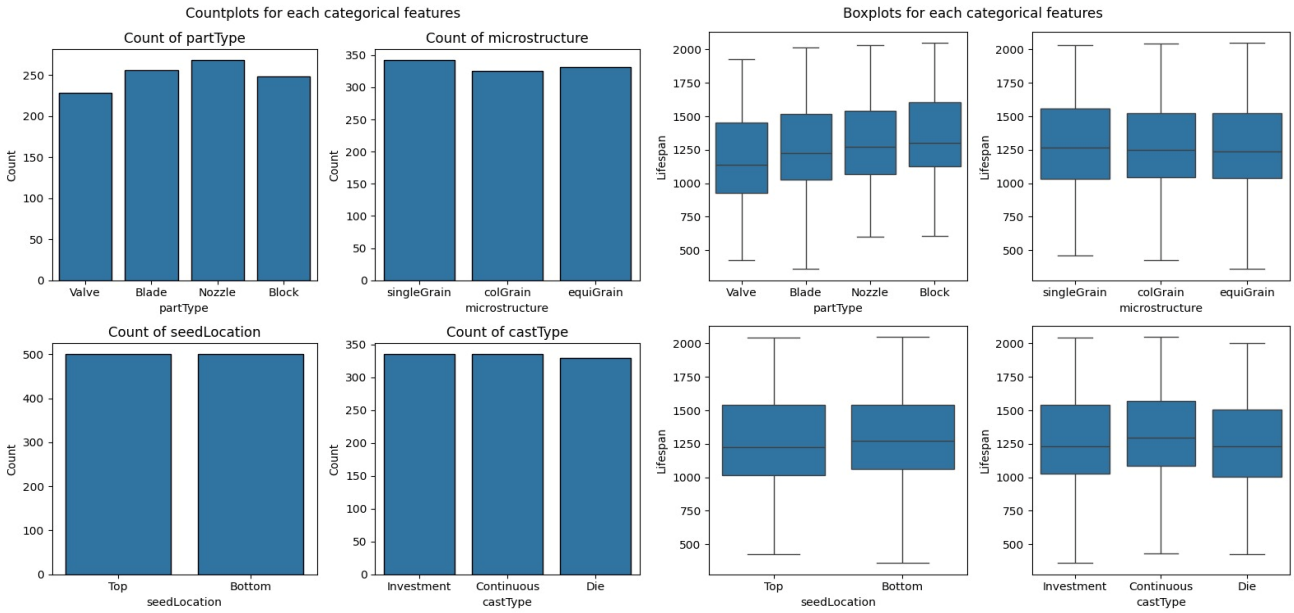

fig- 3: Count plots of categorical columns

Heatmap shows multivariate relationships by annotating the correlation values among the features as shown by fig-4.

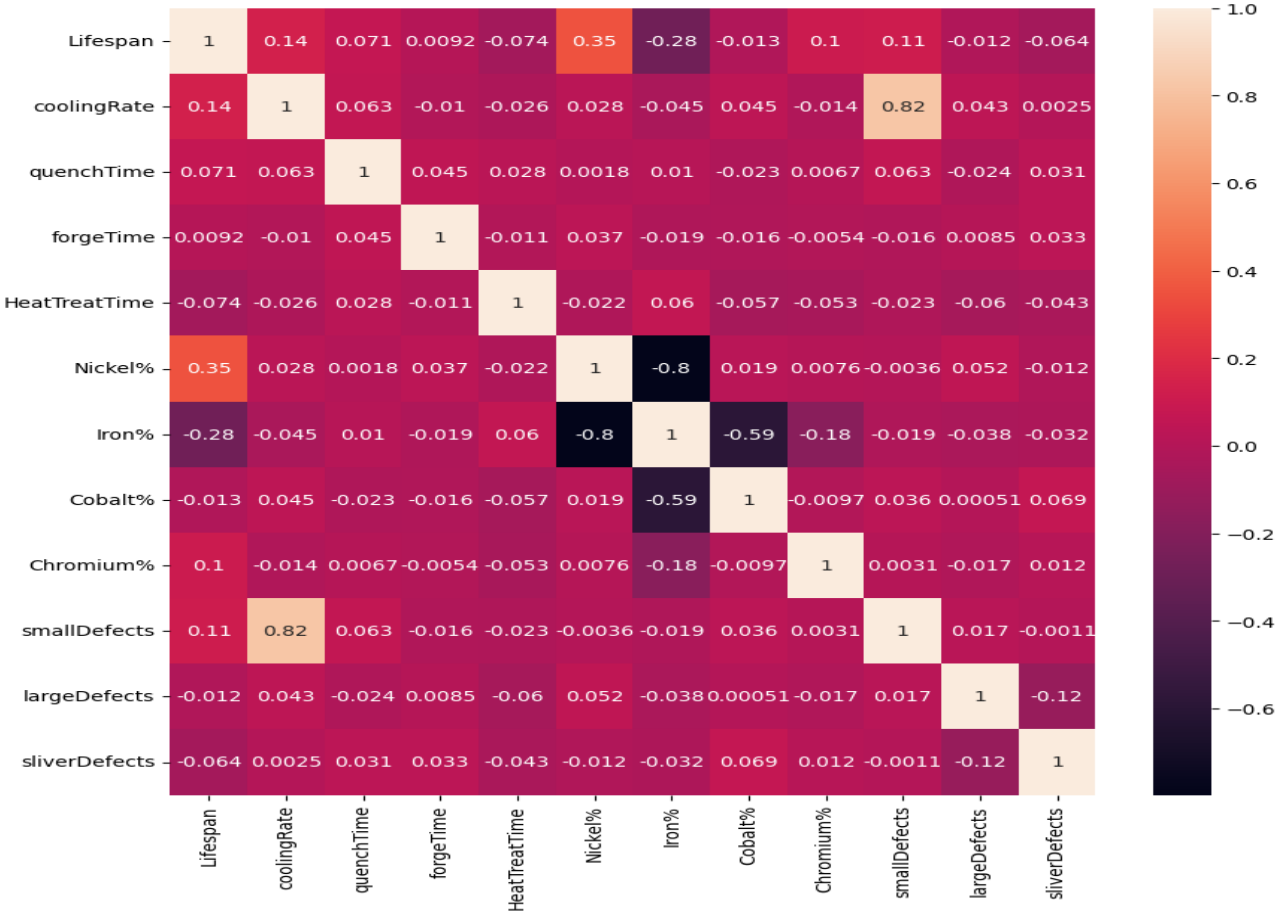

fig- 4: Heatmap of numerical features

Noticeably, for feature selection ‘forgeTime’ had very insignificant correlation value (0.0092) with the target so, it was dropped.

‘Lifespan’ showed a moderate positive correlation with ‘Nickel%’ (0.35) and a moderate negative correlation with ‘Iron%’ (-0.28). Both were strongly negatively correlated with eachother (-0.8), indicating that alloy composition plays a major role in lifespan. Parts with higher Nickel content tended to last longer than those with higher Iron content. This can be confirmed through a scatterplot showed by fig-5.

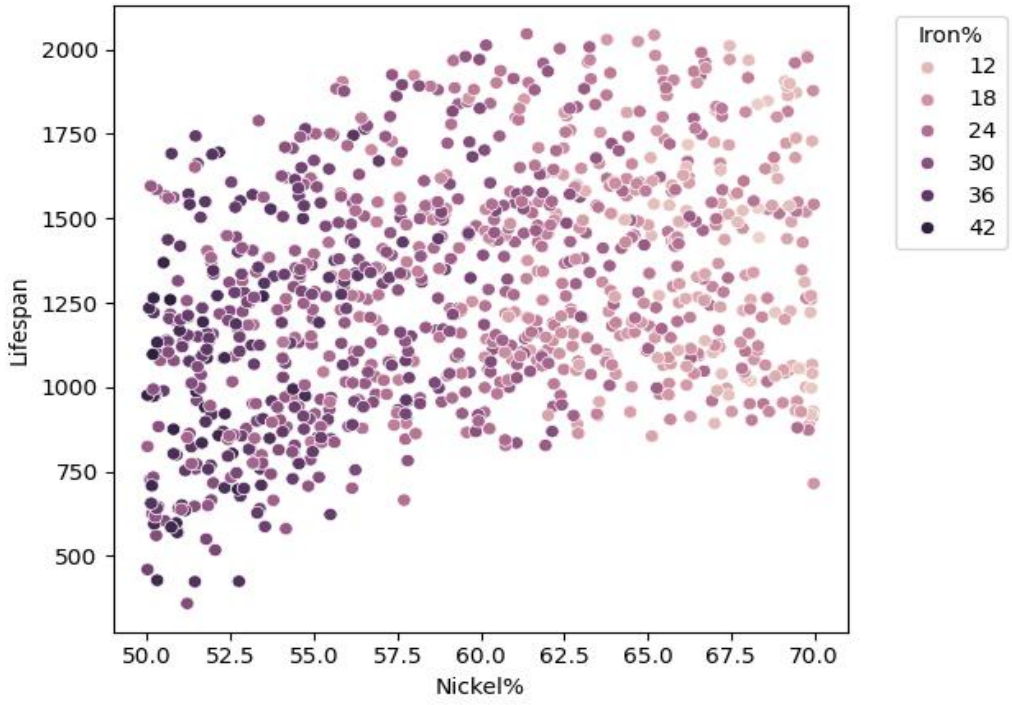

fig- 5: Scatterplot proving alloy composition influence

Because these two features move in opposite directions and convey overlapping information, a new ratio feature, ‘Iron_to_Ni%’ was engineered, dropping original columns. This new ratio feature had a correlation of 0.32 with ‘Lifespan’, suggesting it retains useful predictive information.

Finally, with all the plots and relationships observed so far, there was no strong implication on whether a Regression or Classification model will be suitable. But, the normality of Target is a good sign for Linear Regression models but low correlation with other features hints possible underperformance.

3. Regression Implementation

3.1 Methodology

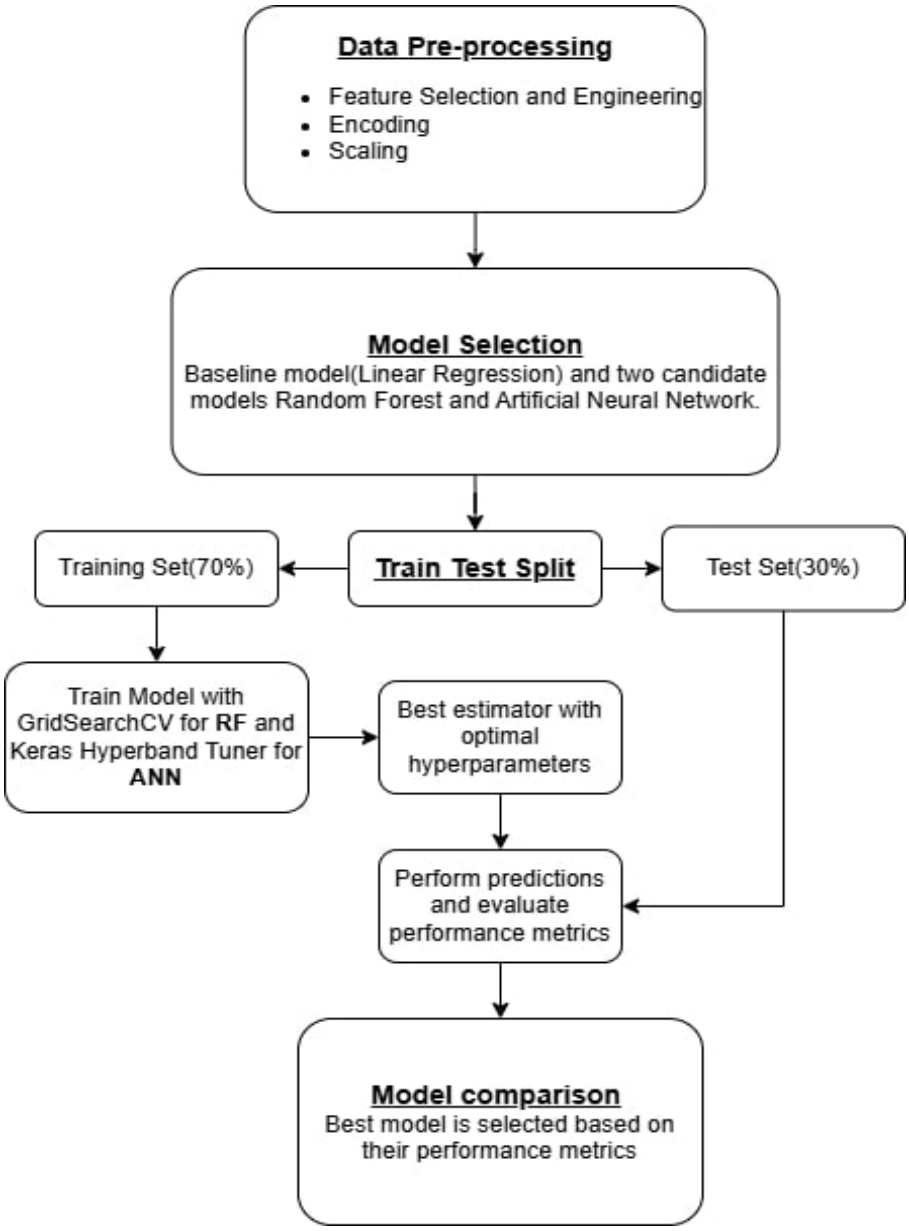

fig- 6*:* Regression Methodology Flowchart

3.1.1 Data Pre-processing

Features were selected and engineered based on EDA, and their observed relationships with the target. A pre-processing pipeline was encompassed with pre-processing steps: Categorical Encoding and Feature Scaling, implemented with the help of sklearn library.

OneHotEncoder() converts categorical data into binary format by creating new columns for each category, marking 1 for presence and 0 for absence in each record.

StandardScaler() transforms data so that each feature has a mean of 0 and a standard deviation of 1, putting everything on the same numerical scale while preserving the relative distances between the data points.

Pipeline() consists of pre-processing methods and selected model instances, and to prevent data leakage, fitted only on the training data.

3.1.2 Model Selection

As baseline model, simple Linear Regression was selected and other two candidate models compared against this baseline were Random Forest (RF) and Artificial Neural Network (ANN).

RF, an ensemble learning method, combines the predictions from multiple decision trees to produce a more accurate and stable prediction. It handles complex, collinearity among data points very well improving generalization, while reducing the risk of overfitting. They are scale-invariant, meaning strong performance even on unscaled data.

ANN on the other hand, uses interconnected nodes called ‘neurons’ stacked as layers which excel in pattern recognition. It learns from the data, continuously improving and adapting to new data without explicitly programming to different scenarios as opposed to traditional algorithms. Since our dataset and problem statement can scale and evolve as data increases, flexibility and adaptability along with the capability to handle complex and dynamic situations of ANNs can prove to be strong. While scaling is not mandatory, doing so ensures faster, more stable convergence and increased model performance.

3.1.3 Train Test Split

The train_test_split() method splits the whole data into training and testing portions as required, which in this case was 70% and 30% respectively.

Training data teaches a model how to behave and testing data assesses how well the model has learned through its performance on unseen data. As the model needs to learn patterns from the data, training set is generally bigger than testing. Testing set needs to be unseen to evaluate the model’s true performance as it has already learned training data.

3.1.4 Experimental Setup and Tuning

For regression models, metrics describing how well the model "fits" the data is vital for which both RMSE (Root Mean Squared Error) and R^2 score often quantify well (Bobbitt, 2021).

a. RMSE: Tells us on average, how far the predictions are from the actual values, in the same scale as the outcome variable. Same scale output helps interpretability and understanding the performance metric. Lower the better.

$$RMSE = \sqrt{\frac{\sum(Predicted_i - Observed_i)^2}{n}}$$

b. R^2: Tells us how well the predictor variables can explain the variation in the response variable. Ranges 0 to 1, higher the better.

$$R^2 = 1 - \frac{\sum(Predicted_i - Observed_i)^2}{\sum\left(Predicted_i - \overline{Observed}\right)^2}$$

For Hyperparameter tuning, GridSearchCV framework tunes Random Forest by exhaustively searching every possible combination within a defined parameter pool and provides the combination that gives the best score. It's computationally costly but reliable. Since our dataset is small, using GridSearchCV makes sense because experimenting with every hyperparameter won't cause computational overload.

For ANN, keras_tuner uses an optimization algorithm called Hyperband that uses a sophisticated strategy of progressive resource allocation to maximize the chances of finding optimal parameters. In this way, it automatically finds the best parameters for the neural network while being computationally efficient.

Hyperparameters considered for these models are in table-1:

Table- 1: Hyperparameter guide

Models	Hyperparameters
Random Forest	1. n_estimators: Number of decision trees. More trees improve accuracy but increase computation time. 2. max_depth: Maximum depth of each decision tree. Limits how complex each tree can become to prevent overfitting.

	3. max_features: How many features to consider when splitting each node. Helps prevent overfitting. 4. min_samples_leaf: Minimum samples needed in each leaf node. Prevents trees from creating nodes with very few samples.
Artificial Neural Network	1. Number of hidden layers: Depth of the neural network. Significantly affects the capacity to learn complex patterns. 2. Neurons per layer: Captures more intricate relationships but increases risk of overfitting. 3. Dropout: Regularization technique which randomly disables neurons during training to prevent overfitting and makes the network robust. 4. Learning Rate: How quickly the network learns from its mistakes, higher means faster updates but risk of overshooting optimal solutions while lower means slower training.

While the ANN was made to run for 50 epochs, a stopping criterion ‘EarlyStopping’ with patience = 10, was defined which monitors the validation score after each epoch and if the score doesn’t improve for 10 consecutive epochs, stops the trial to save time and prevent overfitting.

3.2 Evaluation

For RF, 5-fold cross validation ensures robust training. The best_params_ and best_estimators_ are used to build the final model, which then predicts on the test set and is evaluated using the selected performance metrics.

For ANN, a Sequential model was built using Tensorflow/Keras having an input layer equal to the number of features. Various dense layers and dropout rates were experimented, with a single neuron as the output layer predicting continuous value. Other key elements include ReLU activation, which allows positive values to pass through unchanged while setting all negative values to zero, and the Optimizer which adjusts model's parameter to minimize the loss- in this case, 'RMSE'. 'Adam' an effective optimization algorithm, selects smaller learning rate for frequently updated parameters and larger for parameters of rare features, suitable for optimizing tasks involving both large and small datasets (Hospodarskyy *et al.*, 2024).

Experimentation table-2:

Table- 2: Experimentation guide for Regression models

Models	**Parameters Grid**	**Optimal Hyperparameters**	**Test RMSE Score**	**Test R^2 Score**
Linear Regression (Baseline)			300.39	0.20
Random Forest Regressor	n_estimators: [200, 250, 300, 350], max_depth: [4, 6, 8], min_samples_leaf: [1, 2, 3], max_features: ['log2', 'sqrt']	max_depth = 8, max_features = 'log2', min_samples_leaf = 1, n_estimators = 300	158.70	0.77
Artificial Neural Network	Num of hidden layers: 1 to 4, Num of neurons in each layer min = 32 to max = 512, dropout = 0.0 to 0.5, learning_rate: [0.001, 0.01, 0.1]	Num of hidden layers = 4, Neurons in hidden layer-1 = 192 and dropout = 0, Neurons in hidden layer-2 = 384 and dropout = 0.1, Neurons in hidden layer-3 = 512 and dropout = 0.2, Neurons in hidden layer-4 = 352 and dropout = 0.1,	**147.82**	**0.80**

		Learning rate = 0.01 Early Stopping with patience = 10 but was never triggered		

Comparison table-3 between the models:

Table- 3: Candidate Models Comparison

Models	**Improvement against the Baseline**	**Model Decision**
Linear Regression (Baseline		
Random Forest Regressor	Reduced RMSE by 47% and increased R^2 by 285%	
Artificial Neural Network	Reduced RMSE by 50.79% and increased R^2 by 300%	RMSE **6.9% less** and R^2 **3.9% more** than RF, hence the better model for this task.

Scatterplots in fig-7, support the final model selection showing predicted values(y_pred) against actual values(y_test) made using matplotlib library.

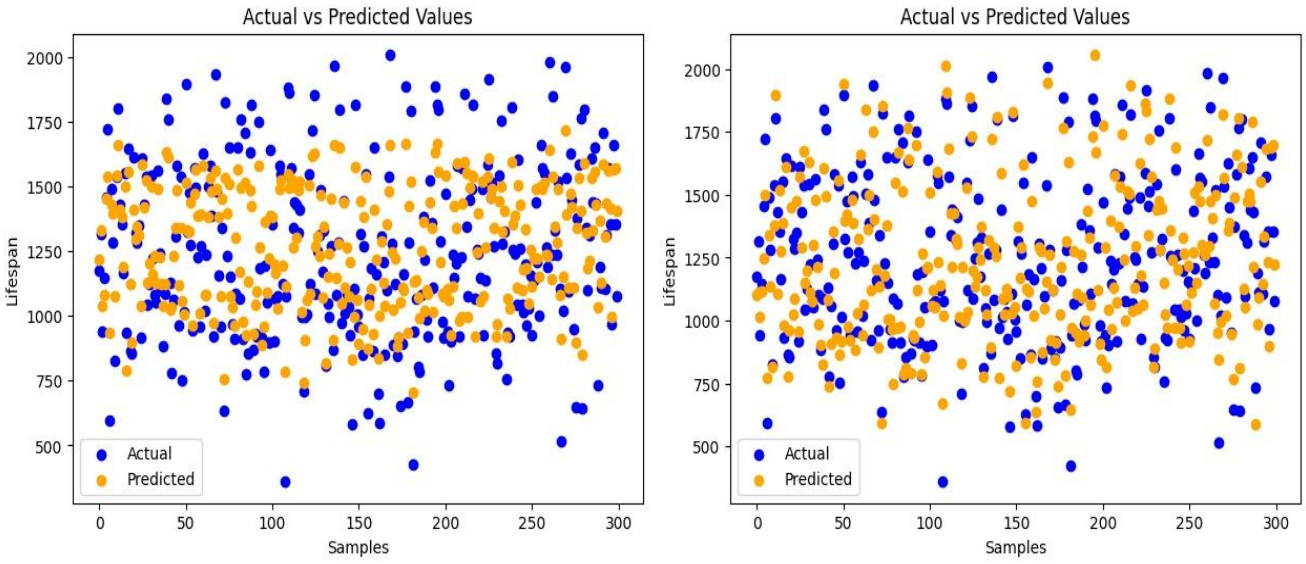

fig- 7: Scatterplots of RF (left) and ANN (right)

3.3 Critical Review

Based on sheer numbers of the performance metrics, ANN comes on top but it’s also important to consider that Random Forest being scale invariant, means less computation and no Feature Scaling step without sacrificing performance. Whether the small performance jump from ANN justifies the extra preprocessing and computation is an important consideration.

Alternatively, a simple L1 or L2 regularized Regression might have produced commendable results with far less complexity and computations.

Pre-defining hyperparameter grids are subjectively selecting the best of the bunch. Hence, there are tuneable parameters like max_leaf_nodes, min_samples_split, etc that missed out for RF.

Similarly for ANN, number of epochs to run is a hyperparameter in itself which hasn’t been experimented more. Also, constricting the number of hidden layers, neurons and choosing the most common optimizer without proper experimentation.

Because the tuning frameworks determine optimal parameters through trials behind the scenes, the final best estimators were provided without showcasing much details of those trials.

4. Classification Implementation

4.1 Feature Crafting

Instead of the default binary thresholding, a hierarchical approach to clustering the data was applied called Agglomerative Clustering, using ‘ward’ for the linkage method, which attempts to minimize the variance between clusters. Hierarchical Clustering was chosen over K-means because K-means excels with data having spherical clusters, but the data is much more spread. It also helps to identify natural and intuitive groupings of datapoints with the perk of reproducibility (*Difference between K means and Hierarchical Clustering*, 2025).

Silhouette score was used to estimate the number of clusters, calculated as (*What is Silhouette Score?*, 2025):

$$s = \frac{b_i - a_i}{\max(a_i, b_i)}$$

where, a_i = intra-cluster distance and b_i = nearest-cluster distance

Dendrogram visualization in fig-8, also portrays that 4 clusters as the best alternative, making this a multi-class classification.

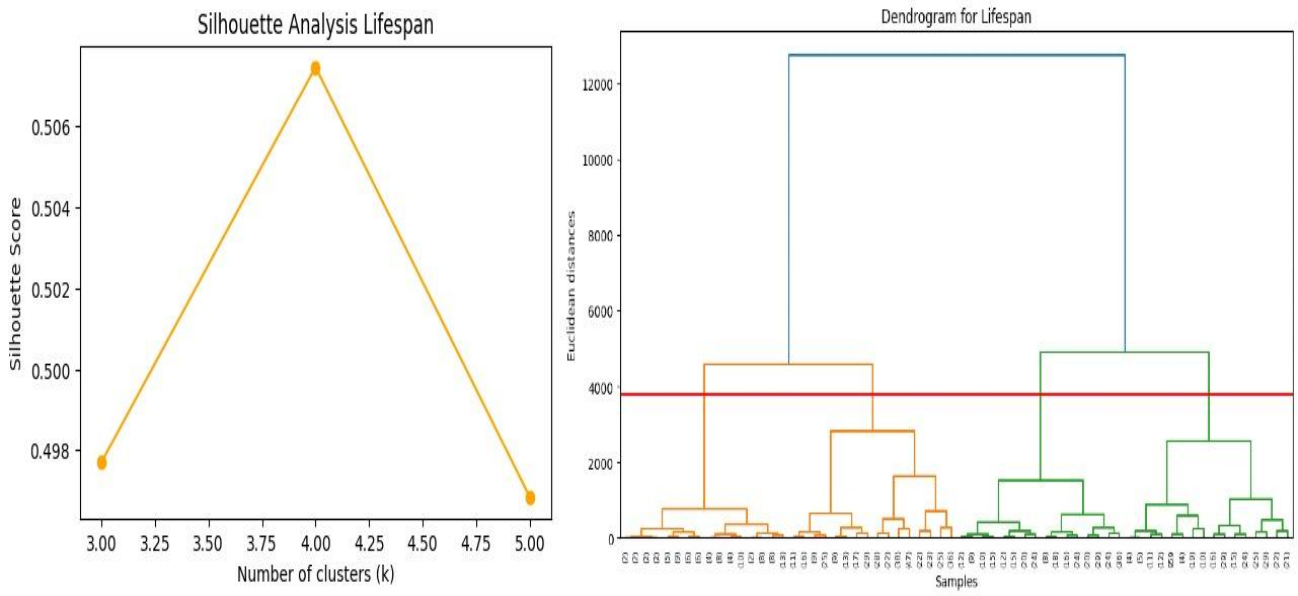

fig- 8: Silhouette score analysis and Dendrogram plot

Grouping into 4 clusters can be considered superior as it provides more granular understanding and representation of continuous phenomena of datapoints. According to binary thresholding, the information conveyed about a part was just whether it was ‘defective’ or not, which is a narrow narrative. This makes a part either completely workable or not at all

with no in-betweens, but if there's an intermediate class as 'At Risk' then, it provides a valuable information of the part on the edge of being 'defective' but not quite yet, so maybe it can be cautiously utilized or maintained to make sure its lifespan persists.

Cluster labels were assigned as per their 'Lifespan' feature as 0: 'Normal', 1: 'Premature', 2: 'At Risk' and 3: 'Well Maintained' using the map() function for interpretation. Resulting dataset had imbalanced class distribution shown by fig-9.

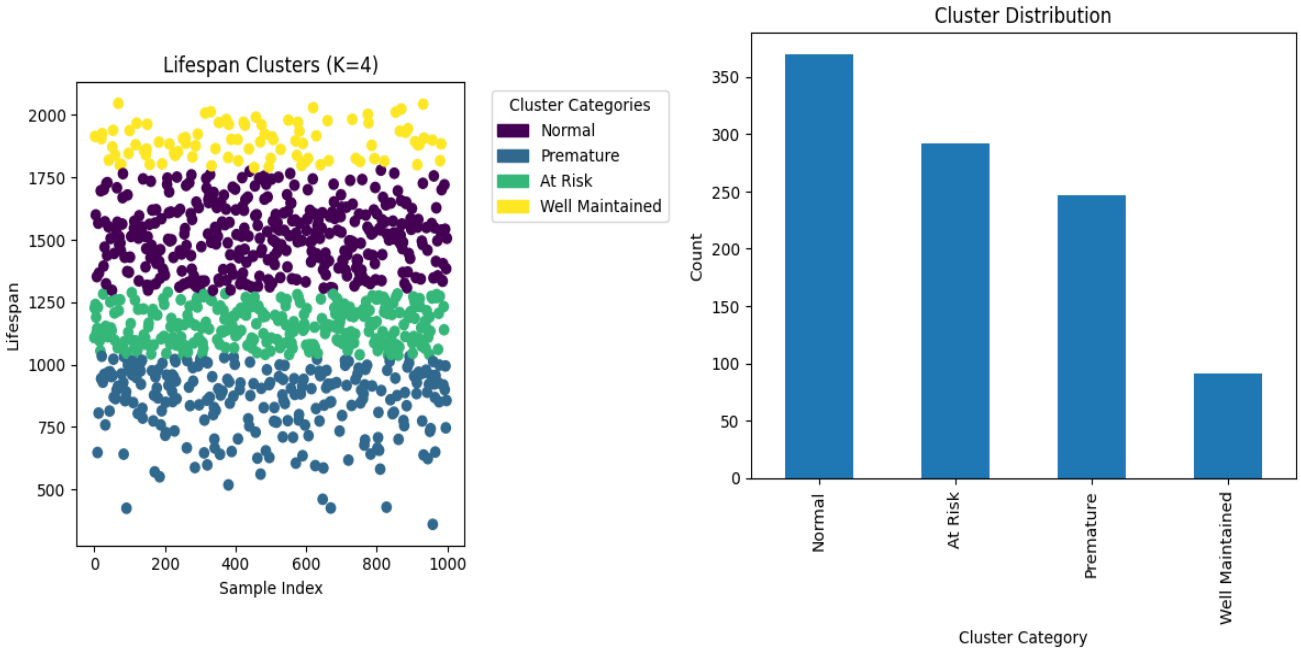

fig- 9: Cluster Category Grouping and Distribution

4.2 Methodology

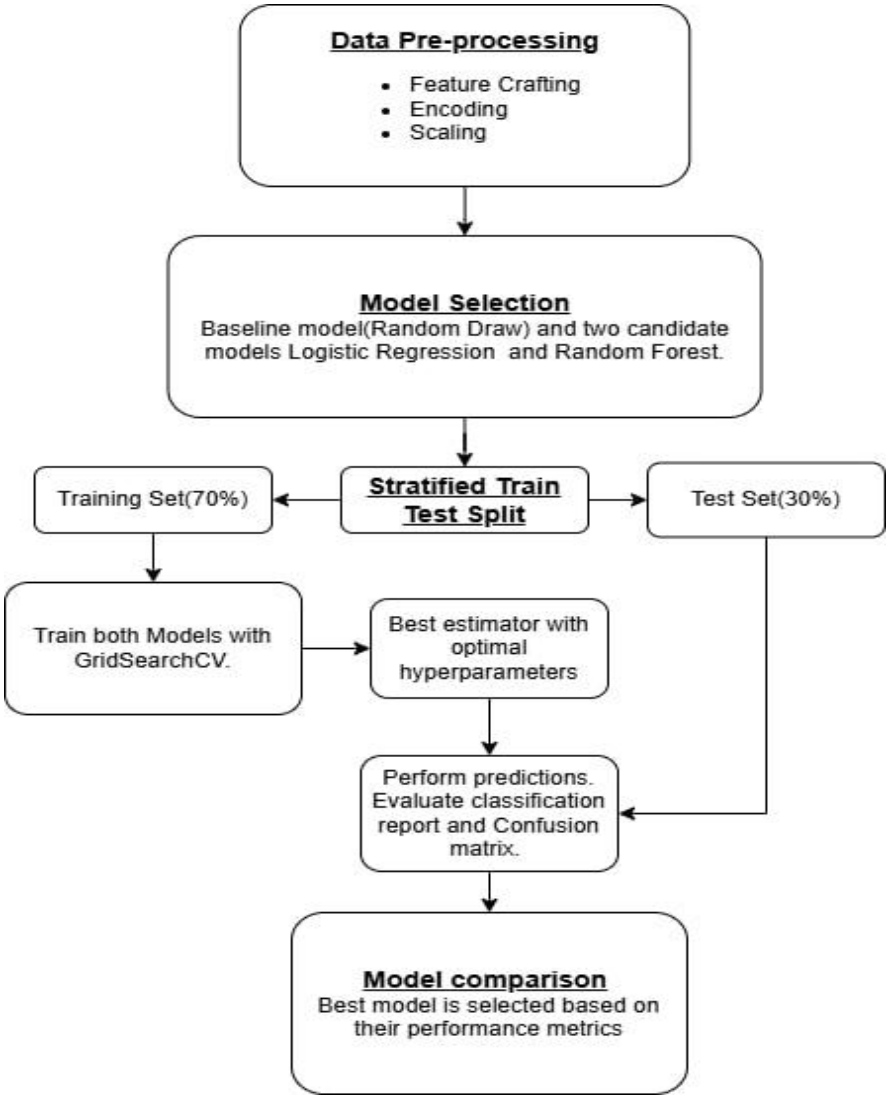

fig- 10: Classification methodology Flowchart

4.1.1 Pre-processing and split

Pre-processing steps followed are feature encoding and scaling methods, same as the regression task integrated with Pipeline(). Stratification was used to ensure that the train and test split accurately portrays the original data distribution, by passing ‘y’ to the ‘stratify’ argument. Split proportion is 70% training and 30% testing but before splitting ‘Lifespan’, ‘life_cluster’ and ‘life_Category’ features are removed from input dataset to prevent data leakage.

4.1.2 Model Selection

For the baseline, a model that randomly draws a class label was made just to quantify and contrast how much a well-defined ML model performs.

As the classification task evolved towards multiclass with our dataset containing no extreme outliers, with sufficient sample size, Logistic Regression became a strong choice with its ability to combine simplicity and efficiency with powerful predictive capabilities. It maintains interpretability even with multiple classes to produce stable results. For multiclass task, 'softmax' function is used as opposed to 'sigmoid' for binary, which outputs the probability of each class and the one with the maximum chosen as the predicted class.

The other model Random Forest Classifier, always stands out as an exceptional choice for multiclass task due to its unique combination of robustness, flexibility and interpretability, working seamlessly with both continuous and categorical features like in our dataset. Additionally, it requires minimal pre-processing and easy to tune parameters, but the best characteristic of this model is its ability to handle imbalanced datasets with uneven class distributions. Being an ensemble model, reduces overfitting risk and provides more stable, reliable predictions.

4.1.3 Experimental Setup and Tuning

Wrong adoption of metrics in this task can cause serious business impact. With imbalanced classes, Accuracy cannot be considered as the main metric since it treats every error with the same weightage but in real world scenarios, some errors are costlier than others. Missing defects can cause costly failures, downtime, warranty claims and safety implications whereas over-flagging parts as defective results in unnecessary rework/testing cost. Therefore, these are the popular metrics for this task (Rajesh, 2024):

Precision measures how many of the items our model predicted as positive were actually positive.

$$Precision = \frac{TP}{(TP + FP)}$$

Recall measures how many of the actual positive items our model correctly identified.

$$Recall = \frac{TP}{(TP + FN)}$$

F1 score can be regarded the best as it measures how well a model balances catching the positives(recall) without making too many false alarms(precision).

$$F1\ score = 2 * \frac{Precision * Recall}{Precision + Recall}$$

Confusion matrix provides n*n matrix for ‘n’ classes mapping actual class against predicted class. Example of a 3*3 matrix:

Confusion Matrix		Predicted Class		
		1	2	3
Actual Class	1	TN	FP	TN
	2	FN	TP	FN
	3	TN	FP	TN

fig- 11: Confusion matrix (Manthena, Jarquín and Howard, 2023, p. 07, Fig. 01)

Where, TP = True Positives, TN = True Negatives, FP = False Positives and FN = False Negatives

Hyperparameter tuning framework GridSearchCV has already been discussed before in the report, as scikit-learn tool that performs cross-validation and tries all possible combinations of parameters to find the optimal configuration. Table-4 shows the hyperparameter guide for Logistic Regression.

Table- 4: Hyperparameter guide for Logistic Regression

Hyperparameter	**Description**
C	Controls the strength of regularization in the model. Higher increases regularization leading to simpler models and lower allowing more complex models.
Solver	Determines the optimization algorithm to find the optimal parameters.

For Random Forest the hyperparameter guide is same as the regression task which can be referred to Table-1.

4.3 Evaluation

Evaluation is always done on the test set so that the real performance of model can be gauged on unseen data. Optimal parameters were chosen from the provided parameter grid using GridSearchCV, reported in table-5:

Table- 5: Experimental guide

Model	**Parameters Grid**	**Optimal Hyperparameters**
Random Draw (Baseline Model)		
Logistic Regression	C: np.linspace(0.001, 1.0, 10), Solver: ["newton-cg", "saga", "lbfgs"]	C: 0.112, Solver: “newton-cg” The ‘newton-cg” solver stands for ‘Newton-Conjugate-Gradient’ which is best for small-medium datasets and allows fast convergence.
Random Forest Classifier	n_estimators: [100, 200, 300], max_depth: [None, 10, 20, 30], min_samples_leaf: [1, 2, 4], max_features: ['log2', 'sqrt']	max_depth = None, max_features = ‘sqrt’, min_samples_leaf = 1, n_estimators = 100

Performance comparison of the respective models across each class with their optimal hyperparameters is shown by table-6:

Table- 6: Performance Comparison

Model		**Precision (per class)**	**Recall (per class)**	**F1 Score (per class)**	**Model Decision**
Baseline	‘Premature’	0.19	0.18	0.18	
	'At Risk'	0.23	0.20	0.22	
	'Normal'	0.39	0.26	0.31	
	'Well Maintained'	0.07	0.22	0.11	

Logistic Regression	‘Premature’	0.45	0.47	0.46	Poor performance only on ‘Well Maintained’ class otherwise better than base model
	'At Risk'	0.36	0.18	0.24	
	'Normal'	0.42	0.66	0.51	
	'Well Maintained'	0.33	0.04	0.07	
Random Forest	‘Premature’	**0.88**	**0.76**	**0.81**	Best performance overall
	'At Risk'	**0.66**	**0.55**	**0.60**	
	'Normal'	**0.67**	**0.94**	**0.78**	
	'Well Maintained'	**1.00**	**0.26**	**0.41**	

Confusion matrices given by Fig-12:

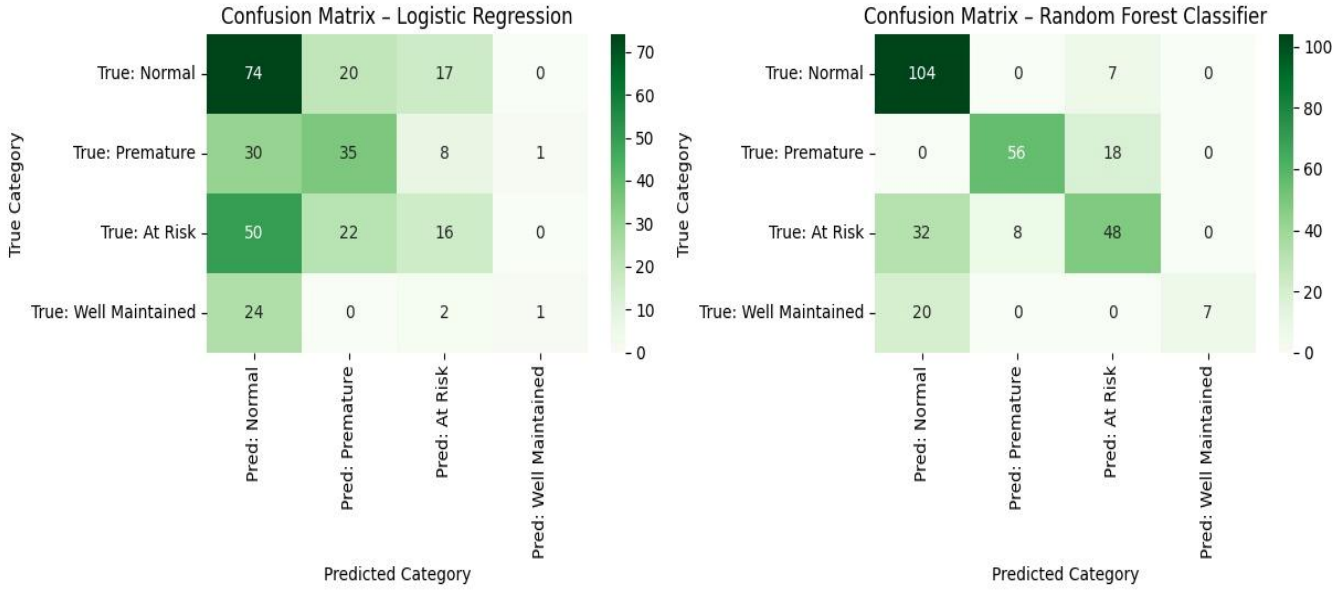

fig- 12: Confusion matrices

In conclusion, Random Forest is the recommended model for this task due to its superiority. Especially, detecting ‘Premature’ defects can provide significant business value and also address safety concerns early. Both candidate models have below par performance for ‘Well Maintained’ class which is understandable as it lacked enough samples due to imbalanced class distribution. Hence, models were not able to be trained much on this class.

4.4 Critical Review

The most important step of this task was crafting cluster labels for datapoints with respect to their lifespan feature. The simple binary approach to thresholding on 1500 hours was deemed naïve hence the need of an alternative grouping, but during experimentation it was quite obvious through dendrogram that grouping into 2 clusters could have been stronger, possibly providing better results.

The problem of class imbalance could have been tackled through strategies like Random Oversampling, SMOTE (saikat, 2021). Lack of training samples for a particular class affected the model's performance for that class.

Logistic Regression model's greatest strength was its greatest weakness, being too simple to catch the complexities within the data. A complex robust model like Neural Networks could have worked better.

A pre-defined parameter pool made the experimentation narrow for searching optimal parameter combination. Several hyperparameters of RF like criterion, min_samples_split, bootstrap and class_weight = 'balanced' could've been explored as well.

5. Conclusions

The ultimate choice of a single model comes down to the use-case and requirements of the manager. The key business decision is:

“Is this part safe to use, or must it be scrapped?”. This is fundamentally a go / no-go decision.

Conclusively, a classification model provides a simpler, clear actionable output which has higher communication value, while also being easy to understand and justify for a non-technical audience. It places the output among several interpretable classes providing a clear information about that metal part.

The manufacturing line doesn’t need the precise lifespan value, it needs to know whether a part meets the minimum requirement of be considered usable or not. The main underlying problem of this task is defect-driven, not smooth lifespan prediction.

Proper classification also helps in manufacturing and engineering value by providing diversified classes for engineers to study the characteristics and uncover relevant patterns of the parts associated with those classes.

Regression model on the other hand, provides quantitative output which is non-interpretable without context and questionable for decision making.

My final recommendation is the Random Forest Classification model, a simple robust and interpretable model whose performance scales with data while being computationally-efficient. It has superior all-round performance showcasing model’s predictive power and output relevant to the business problem and the task of the coursework.

6. References

Bobbitt, Z. (2021) 'RMSE vs. R-Squared: Which Metric Should You Use?', *Statology*, 22 June. Available at: https://www.statology.org/rmse-vs-r-squared/ (Accessed: 10 November 2025).

Hospodarskyy, O. *et al.* (2024) 'Understanding the adam optimization algorithm in machine learning', *2024* [Preprint].

Islam, M.T. (2023) 'Managing Skewness in Zero-Valued Columns: Strategies and Techniques', *Medium*, 6 June. Available at: https://zahin178.medium.com/managing-skewness-in-zero-valued-columns-strategies-and-techniques-ee5be108d0fb (Accessed: 7 November 2025).

saikat (2021) '5 Techniques to Handle Imbalanced Data For a Classification Problem', *Analytics Vidhya*, 21 June. Available at: https://www.analyticsvidhya.com/blog/2021/06/5-techniques-to-handle-imbalanced-data-for-a-classification-problem/ (Accessed: 17 November 2025).

Manthena, V., Jarquín, D. and Howard, R. (2023) 'Integrating and optimizing genomic, weather, and secondary trait data for multiclass classification', *Frontiers in Genetics*, 13, p. 1032691. Available at: https://doi.org/10.3389/fgene.2022.1032691.

Rajesh, S. (2024) 'Confusion Matrix for Multiclass Classification', *Medium*, 28 April. Available at: https://medium.com/@sreenilarajesh/confusion-matrix-for-multiclass-classification-fe7b6901a541 (Accessed: 28 November 2025).

Difference between K means and Hierarchical Clustering (2025) *GeeksforGeeks*. Available at: https://www.geeksforgeeks.org/machine-learning/difference-between-k-means-and-hierarchical-clustering/ (Accessed: 1 December 2025).

What is Silhouette Score? (2025) *GeeksforGeeks*. Available at: https://www.geeksforgeeks.org/machine-learning/what-is-silhouette-score/ (Accessed: 1 December 2025).

Declaration of AI Use

I have used AI while undertaking my assignment in the following ways:

- To develop research questions on the topic – YES
- To create an outline of the topic – YES
- To explain concepts – YES
- To support my use of language – YES